ARTIFICIAL INTELLIGENCE

TOPIC 5.5

Generative models

Explanatory study notes

Theory, worked examples and revision questions

What you should be able to do

Distinguish prediction from generation; explain deterministic autoencoders, VAEs, GANs and diffusion; interpret reconstruction, KL and adversarial losses; recognize mode collapse; explain conditioning and latent diffusion; carry out a short reconstruction or noise calculation.

Based on the teacher PDFs supplied in `Lecture Slides`. The notes explain and reorganize that material. Worked calculations are study examples derived from the slide concepts, unless explicitly identified as a slide exercise. Clarifications of ambiguous or incorrect expressions are marked in context. References use **PDF page numbers**, counted from 1, rather than printed slide numbers.

English edition • 24 October 2026

Contents

1 Discriminative and generative modeling **3**

2 Autoencoders: learn to reconstruct **3**

2.1 Why the bottleneck matters . . . 3

2.2 Small reconstruction and count examples . . . 4

2.3 Why arbitrary latent sampling can fail . . . 4

3 Variational autoencoders **4**

3.1 Reconstruction plus regularization . . . 4

3.2 Reparameterization . . . 5

3.3 A small KL calculation . . . 5

4 GANs: a learned adversarial signal **5**

4.1 Alternating optimization . . . 5

4.2 Generator losses and interpretation . . . 6

5 GAN difficulties and image translation **6**

5.1 Mode collapse and unstable training . . . 6

5.2 CycleGAN . . . 6

6 Diffusion: learn to reverse a noise process **6**

6.1 Forward noising . . . 7

6.2 Training the denoiser . . . 7

6.3 Sampling . . . 7

7 Conditioning and latent diffusion **7**

8 Compare the model families **8**

9 Place these models in the course **8**

10 Revision questions and answers **9**

10.1 Questions . . . 9

10.2 Answers . . . 9

11 Source guide **10**

1 Discriminative and generative modeling

Teacher slides: 5.5. Generative Models, pp. 3–6.

A discriminative classifier models a decision rule or a conditional distribution such as $p(y \mid x)$: given an observation, predict its label. A generative model represents how observations can arise, for example through $p(x)$, $p(x \mid y)$ or a joint distribution $p(x, y)$.

In the classical classification formulation, a generative approach can model class priors $p(y)$ and likelihoods $p(x \mid y)$, then use Bayes' rule to obtain $p(y \mid x)$. Modern generative models can also model observations without class labels at all.

Training data supplies samples from an unknown distribution. The aim is to learn enough structure to sample plausible new observations. Memorizing a few training examples is not the same as adequately modeling the distribution's diversity.

Generation does not require an explicit tractable density

Some models define tractable probabilities; others generate samples without providing an easily evaluated likelihood. A GAN generator is a key example of the latter. The general capability to generate samples should not be confused with having a closed-form density for every input.

An autoregressive language model is also generative: it models a sequence through successive token probabilities. This chapter emphasizes image-oriented examples, but generation applies to text, sound, video and other data.

2 Autoencoders: learn to reconstruct

Teacher slides: 5.5. Generative Models, pp. 7–8.

An autoencoder has an encoder f_θ and decoder g_ϕ:

$$z = f_\theta(x), \qquad \widehat{x} = g_\phi(z).$$

It is trained so that $\widehat{x}$ resembles x, for example by minimizing mean squared reconstruction error:

$$L_{\text{rec}} = \frac{1}{D} \sum_{j=1}^{D} (x_j - \widehat{x}_j)^2.$$

The target is the original input, so no externally supplied semantic class label is needed. A low-dimensional bottleneck or other restrictions encourage a useful representation rather than unrestricted copying.

2.1 Why the bottleneck matters

With enough unconstrained capacity, an autoencoder can learn an identity-like mapping with little useful compression. Restricting latent dimension, adding regularization or using a denoising task can encourage structure. A compressed representation is not automatically the best representation for every downstream task.

An encoder–decoder for segmentation also maps through intermediate features, but its target is a semantic mask rather than the input image. Similar architecture does not imply the same objective or supervision.

2.2 Small reconstruction and count examples

For $x = (1, 3)$ and $\hat{x} = (2, 1)$, the sum of squared errors is $1 + 4 = 5$ and the mean is $5/2 = 2.5$. State whether the loss uses a sum or mean.

A dense autoencoder $8 \to 3 \to 8$ with one bias per unit has

$$3(8 + 1) + 8(3 + 1) = 27 + 32 = 59$$

parameters. Its latent vector has three entries, but its learned model has 59 coefficients. Latent dimension, parameter count and number of training samples are different quantities.

2.3 Why arbitrary latent sampling can fail

A deterministic autoencoder learns to decode the latent codes that its encoder produces on the data. It does not necessarily arrange the whole latent space to match a simple sampling distribution. Random Gaussian latent vectors may land in poorly trained regions and decode badly.

The decoder can mathematically accept any latent vector of the correct shape. The slide's warning is about output quality outside the learned code distribution, not that decoding those vectors is computationally impossible.

3 Variational autoencoders

Teacher slides: 5.5. Generative Models, p. 8.

A variational autoencoder (VAE) encodes an input into a distribution over latent vectors, often a diagonal Gaussian:

$$q_\phi(z \mid x) = \mathcal{N}\Big(\mu_\phi(x), \operatorname{diag}(\sigma_\phi(x)^2)\Big) .$$

The encoder predicts mean and scale parameters, not a single fixed code. A decoder models observations conditional on a sampled latent. A common prior is $p(z) = \mathcal{N}(0, I)$.

3.1 Reconstruction plus regularization

A standard negative-ELBO objective has the form

$$L(x) = \underbrace{\mathrm{E}_{q_\phi(z|x)}[-\log p_\theta(x \mid z)]}_{\text{reconstruction term}} + \underbrace{D_{\mathrm{KL}}(q_\phi(z \mid x) \| p(z))}_{\text{prior regularization}} .$$

The reconstruction term rewards retaining information needed to reproduce the input. The KL term penalizes deviations of each encoded distribution from the prior, encouraging a latent space compatible with sampling. Squared-error reconstruction corresponds to a particular Gaussian observation model and scaling, rather than being the only possible reconstruction loss.

Increasing the relative weight of the KL term changes a trade-off; it does not guarantee improvements in both reconstruction and sampling. Excessive pressure can make the latent carry little information about the input.

3.2 Reparameterization

Write a sample as

$$\epsilon \sim \mathcal{N}(0, I), \qquad z = \mu + \sigma \odot \epsilon.$$

Randomness is supplied by ϵ, while the path through μ and σ is differentiable. For one coordinate with $\mu = 1, \sigma = 2, \epsilon = -0.5$, the sampled latent is $z = 0$.

3.3 A small KL calculation

For a diagonal Gaussian versus $\mathcal{N}(0, I)$,

$$D_{\mathrm{KL}} = \frac{1}{2} \sum_j \left(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1 \right).$$

For one coordinate with $\mu = 1, \sigma = 1$, this is $\frac{1}{2}(1 + 1 - 0 - 1) = 0.5$ nats. With $\mu = 0, \sigma = 1$, it is zero because the two distributions match. Here σ is a standard deviation; σ^2 is a variance.

During reconstruction, sample from the encoder's distribution for a given input. During unconditional generation, sample z from the prior and decode it. A structured latent objective makes this sampling procedure meaningful, but does not guarantee realistic outputs for every point in the infinite prior support.

4 GANs: a learned adversarial signal

Teacher slides: 5.5. Generative Models, pp. 9–13.

A generative adversarial network trains two networks:

- The generator G maps a random input z to a synthetic sample $G(z)$.
- The discriminator D learns to distinguish real training examples from generated ones.

The discriminator supplies a learned signal about realism. The generator adjusts its outputs so the discriminator is more likely to regard them as real.

In the original minimax formulation,

$$\min_G \max_D \left[\mathrm{E}_{x \sim p_{\mathrm{data}}} \log D(x) + \mathrm{E}_{z \sim p(z)} \log(1 - D(G(z))) \right].$$

Read the roles rather than memorizing a long derivation: D seeks high scores for real samples and low scores for fake samples, while G changes the fake samples to oppose that distinction.

4.1 Alternating optimization

When updating D, treat generated samples as examples labeled fake and real data as examples labeled real. When updating G, hold the discriminator's parameters fixed but differentiate through its computation to the generated image and generator.

Fixed discriminator parameters do not mean no gradient can pass through the discriminator's input. That gradient is precisely how the generator can learn.

The real/fake labels are constructed from the data source, not human semantic labels such as dog or cat. Thus the slide's description as weak supervision should not be taken to require manually labeled object categories for an unconditional GAN.

4.2 Generator losses and interpretation

A commonly used non-saturating generator loss is

$$L_G = -\,\mathrm{E}_z \log D(G(z)).$$

For a fixed generated sample, raising $D(G(z))$ from 0.2 to 0.8 reduces this loss from about 1.609 to 0.223. The objective explicitly encourages generated samples that the discriminator regards as real.

At the idealized distribution-matching equilibrium with balanced real/fake sampling, the optimal discriminator returns $1/2$. This means it cannot distinguish the two distributions, not that each generated image is only half correct. In actual training, $D \approx 1/2$ alone is not proof of success: an untrained or ineffective discriminator can also be uncertain.

5 GAN difficulties and image translation

Teacher slides: 5.5. Generative Models, pp. 13–16.

5.1 Mode collapse and unstable training

Mode collapse means the generator produces too little diversity: many different latent inputs map to the same or a small set of outputs that fool the current discriminator. Individual outputs may look realistic while important modes of the data distribution are absent.

Generator and discriminator objectives change as the other model changes. This can cause oscillations and imbalance. A very strong discriminator can produce weak generator gradients under some losses, especially the original saturating formulation. Alternative losses and training strategies address parts of these problems; no single discriminator score diagnoses all of them.

5.2 CycleGAN

CycleGAN illustrates unpaired translation between domains A and B. It learns mappings $G : A \to B$ and $F : B \to A$ with adversarial losses that encourage target-domain realism and cycle-consistency terms such as

$$\|F(G(x)) - x\|_1.$$

The cycle encourages translated content to support reconstruction of the original. Unpaired means a training image in A need not have an exact corresponding target image in B. Cycle consistency does not by itself uniquely determine the intended semantic mapping.

Style transfer and other image-editing tasks can use different losses and architectures; not every image stylization method is a GAN. The important connection is that a learned realism or perceptual signal can complement a direct pixelwise objective.

6 Diffusion: learn to reverse a noise process

Teacher slides: 5.5. Generative Models, pp. 17–20.

6.1 Forward noising

The forward process gradually adds Gaussian noise to a clean observation x_0 until the result is close to noise. A standard formulation writes

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I),$$

where $\bar{\alpha}_t$ expresses how much original signal remains at step t. The schedule is specified rather than learned by a discriminator.

For a toy scalar with $\bar{\alpha}_t = 0.64$, $x_0 = 2$ and a chosen noise sample $\epsilon = 1$, we get

$$x_t = 0.8(2) + 0.6(1) = 2.2.$$

A noisy value can be larger than the clean value: noise is a random perturbation, not simply a deterministic shrinking of every pixel.

6.2 Training the denoiser

A common objective trains a network $\epsilon_\theta(x_t, t, c)$ to predict the sampled noise from a noisy example, its timestep and optional conditioning c:

$$L_{\text{noise}} = \mathrm{E}\left[\|\epsilon - \epsilon_\theta(x_t, t, c)\|^2\right].$$

Because the training procedure generated ϵ, it knows the target without a human labeling the noise. For true scalar noise 0.3 and prediction 0.1, squared error is 0.04.

The timestep tells the network how noisy the input is expected to be. Training commonly samples a timestep and constructs the corresponding noisy input directly; it does not require a complete sequential reverse-generation trajectory for every training example. Other parameterizations predict the clean signal or related quantities, but the central idea is learning a denoising task at different noise levels.

6.3 Sampling

Start with noise and iteratively apply a learned reverse process to obtain a structured sample. The exact update and any added sampling noise depend on the sampler. This is a learned statistical reversal, not a deterministic inversion that retrieves one original training image.

Standard many-step generation can require more network evaluations than a single GAN or VAE decoder pass. Fewer-step samplers and other improvements change that trade-off, so avoid treating one fixed step count as a universal requirement.

7 Conditioning and latent diffusion

Teacher slides: 5.5. Generative Models, pp. 5, 20–21; 5.3. Foundation models, pp. 24–25.

A conditional generator models observations given information such as a label, text prompt, image or other control. The same random seed can produce a different result under different conditioning; different seeds can produce different results for the same prompt.

In latent diffusion, an autoencoder maps images to a compressed latent space. Diffusion runs on those latents, and the decoder converts the final denoised latent back to an image:

$$\text{prompt} \to \text{text encoder} \to \text{conditioning}, \qquad \text{latent noise} \to \text{denoiser} \to \text{image decoder}.$$

The text representation can enter through attention or another conditioning mechanism. A text encoder represents language; the denoiser learns the generative transformation; the image decoder maps representations to pixels. These components have different jobs.

Working on a smaller spatial representation can reduce denoising cost. It does not eliminate the information bottleneck or guarantee that small details survive compression. The deck shows both U-Net-style and transformer-based denoising ideas in the broader course context.

8 Compare the model families

Teacher slides: 5.5. Generative Models, p. 22.

Family	Core objective	Sampling/use	Common limitation
AE	Reconstruct the input through a representation.	Encode and decode observed data.	Arbitrary random latent codes need not decode plausibly.
VAE	Probabilistic reconstruction plus latent prior regularization.	Sample from a prior and decode.	Reconstruction, latent information and sample quality trade off.
GAN	Generator competes with a real/fake discriminator.	Usually one generator pass from noise.	Adversarial instability and mode collapse.
Diffusion	Learn denoising across noise levels.	Usually iterative denoising from noise.	Sampling can require many network evaluations.

Autoencoders and VAEs generally use direct differentiable losses without an adversarial opponent. Diffusion noise regression also avoids the basic GAN minimax game. That is a useful training distinction, not a promise that any particular model is effortless to train.

Image quality is not one number: realism, diversity, reconstruction fidelity and prompt agreement are different criteria. A crisp but repetitive generator can have mode collapse; a good reconstruction model may be poor at random sampling. No unconditional ranking in the slide comparison table can replace specifying the task and evaluation.

9 Place these models in the course

Teacher slides: 5.5. Generative Models, pp. 25–26.

Artificial intelligence includes methods beyond learning, such as search and planning. Machine learning includes methods beyond neural networks, such as k-NN, SVMs and random forests. Deep learning uses multilayer neural models.

CNN, RNN and transformer describe architectural families. GAN and diffusion describe generative modeling/training frameworks that can use different neural components. NLP, computer vision

and robotics are application areas, not mutually exclusive model architectures. For example, a transformer denoiser can be part of a diffusion system for computer vision.

10 Revision questions and answers

10.1 Questions

1. What is the difference between modeling $p(y \mid x)$ and generating samples of x?
2. What target trains a basic autoencoder?
3. For $x = (1, 3)$ and $\widehat{x} = (2, 1)$, what is mean squared reconstruction error?
4. Why might a randomly sampled latent decode badly in a deterministic autoencoder?
5. What does a VAE encoder output, and what does its KL term encourage?
6. For $\mu = 1, \sigma = 2, \epsilon = -0.5$, compute the reparameterized latent.
7. Which GAN network generates samples, and which evaluates real versus fake?
8. During a generator update, can gradients pass through a discriminator whose parameters are fixed?
9. What is mode collapse?
10. Does a diffusion model learn the prescribed forward noise schedule or a reverse denoising task?
11. Where does the target noise come from in noise-prediction training?
12. In latent diffusion, what do the text encoder, denoiser and image decoder each do?

10.2 Answers

1. The former predicts labels conditional on an observation; the latter models how to produce observations, possibly with conditioning.
2. The original input, compared with its reconstruction.
3. $(1 + 4)/2 = 2.5$.
4. The encoded data need not fill or match the chosen random sampling distribution.
5. A distribution's parameters, commonly mean and scale; KL encourages compatibility with a chosen prior.
6. $1 + 2(-0.5) = 0$.
7. The generator G and discriminator D, respectively.
8. Yes; gradients with respect to the generated input must pass through D to train G.
9. Insufficient generated diversity, with many latent inputs producing a narrow set of outputs.
10. It learns the reverse denoising task; the forward noising process supplies training examples.
11. The random noise sampled by the training procedure is known.
12. Text features condition generation; the denoiser removes latent noise iteratively; the decoder maps the resulting latent to pixels.

11 Source guide

`5.5. Generative Models.pdf`: modeling distinctions (pp. 3–6), AE/VAE (pp. 7–8), GAN objectives and difficulties (pp. 9–13), translation/style applications (pp. 14–16), diffusion (pp. 17–21), comparison (p. 22), and the course hierarchy (pp. 25–26). The VAE/KL, GAN-loss and diffusion equations make the slide concepts explicit; the numerical mini-examples are study applications, not additional teacher-set problems.